

Functions:

A function is a named block of reusable code that performs a specific task.

A function in Python is a reusable block of code that performs a specific task. It is defined using the `def` keyword and executed by calling its name. Functions help reduce code duplication, improve readability, and make programs easier to maintain. Instead of writing the same code multiple times, we write it once inside a function and call it whenever needed."

Why Do We Use Functions?

-  Avoid writing the same code again and again.
-  Make programs easier to read.
-  Easier to fix bugs (change the code in one place).
-  Reuse the same logic multiple times.

How a Function Works

Step 1: Define the function

```
def greet():  
    print("Hello")
```

This **creates** the function.

Nothing is printed yet.

Step 2: Call the function

```
greet()
```

Now Python executes everything inside the function.

Output:

Hello

Real-Life Example

Think of a **TV remote**.

You press the **Power** button.

Inside the TV, hundreds of instructions run.

You don't know all those instructions.

You just press:

Power Button

A function is the same.

You call:

`greet()`

Python executes all the instructions inside `greet()`.

Decorators

A decorator is a function that adds extra functionality to another function without changing its original code.

Scenario

You have **100 functions**, and before every function runs, you want to print:

"Function started"

Instead of writing `print("Function started")` inside every function, you create **one decorator**.

Code

```
def start_message(func):  
    def wrapper():  
        print("Function started")  
        func()  
    return wrapper
```

```
@start_message  
def login():  
    print("Login successful")
```

```
@start_message  
def logout():  
    print("Logout successful")
```

```
login()  
logout()
```

Output

```
Function started  
Login successful  
Function started  
Logout successful
```

How it works

- `start_message` is the **decorator**.
- It receives the original function (`login` or `logout`).
- The `wrapper()` function runs first.
- It prints "Function started".
- Then it calls the original function using `func()`.
- The same decorator can be reused for any number of functions.

To Remember

Think of a **gift box**.

- 📁 **Gift** = Original function
- 📁 **Gift wrapper** = Decorator

The gift remains the same, but the wrapper adds something extra.

Arbitrary Arguments - *args

Definition:

*args allows a function to accept **any number of positional arguments**.

args is a tuple.

Arbitrary Keyword Arguments - **kwargs

```
def display(*args, **kwargs):  
    print("Args:", args)  
    print("Kwargs:", kwargs)
```

```
display(10, 20, 30, name="Manoj", city="Chennai")
```

****kwargs is a dictionary.**

Recursion

Recursion is a technique where a function calls itself repeatedly to solve a problem.

Every recursive function must have a base case to stop the recursion; otherwise, it will continue calling itself.

Python Generators

Simple Definition

A generator is a special type of function that returns values one at a time using the yield keyword instead of returning all values at once.

Why use generators?

Normally, a function returns all the data at once.

```
def numbers():  
    return [1, 2, 3, 4, 5]
```

```
print(numbers())
```

Output

```
[1, 2, 3, 4, 5]
```

The entire list is created in memory.

With a generator:

```
def numbers():
```

```
    i=[1, 2, 3, 4, 5]
```

```
        yield i  
        yield 2  
        yield 3  
        yield 4  
        yield 5
```

```
gen = numbers()
```

```
print(next(gen))
print(next(gen))
print(next(gen))
```

Output

```
1
2
3
```

Here, the values are produced **one at a time**, only when needed.

How `yield` works

When Python reaches `yield`:

- It returns the value.
- It pauses the function.
- The next time `next ( )` is called, it continues from where it stopped.

Student Marks Management

Create a program that:

- Takes any number of student marks using
- Calculates total and average.
- Uses `if` to print: each subject
 - Pass (≥ 35)
 - Fail (< 35)
- Use a decorator to print: decorator
 - "Calculation Started"

- "Calculation Finished"

Mini Banking Application

Requirements

Account_balance = 10000

1. Deposit
2. Withdraw
3. Check Balance

Rules

- Use variables to store balance.
- Use functions for each operation.

Use a decorator to print:

Transaction Started

- Transaction Completed
- Use `if-else` to validate withdrawals (insufficient balance).
- Use loops to keep showing the menu until the user exits.
- Add a function using `**kwargs` to display account details (e.g., name, account_no, branch).